



img/Escudo_UNC.png

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

CARRERA INGENIERÍA ELECTRÓNICA

**PROYECTO INTEGRADOR PARA LA OBTENCIÓN DEL
TÍTULO DE GRADO INGENIERO ELECTRÓNICO**

**“DISEÑO, DESARROLLO E IMPLEMENTACIÓN DE UN
SOFTWARE PARA LA ADQUISICIÓN Y ANÁLISIS DE ONDAS
DE IMPULSO TIPO RAYO (1,2/50 μ s).”**

Alumno

Tardón, Damián David

Director

Ing. Blasco, Marcos Javier

Co-Director

Ing Serra, Gabriel Horacio

Córdoba, República Argentina

Abril / 2026

Capítulo 1

Implementación del motor de procesamiento (DSP)

1.1 Algoritmo de cálculo de parámetros

El motor de cálculo implementado determina los parámetros característicos de la onda de impulso atmosférico tipo rayo (1, 2/50 μ s) bajo los lineamientos de la norma IEC 60060-1. El procesamiento se ejecuta sobre la curva de tensión de ensayo $u_t(t)$, definida como la suma de la curva base $u_m(t)$ y la curva residual filtrada $r_f(t)$.

El valor cresta de la tensión de ensayo (U_t) se determina como el máximo valor absoluto del vector resultante $u_t(t)$, conservando el signo correspondiente a la polaridad del impulso analizado.

$$U_t = \max(|u_m(t) + r_f(t)|) \cdot \text{sgn}(U_e) \quad (1.1)$$

Donde:

U_t : valor cresta de la tensión de ensayo.

$u_m(t)$: vector de la curva base ajustada.

$r_f(t)$: vector de la curva residual filtrada.

U_e : valor extremo de la curva registrada compensada en *offset*.

La norma define el tiempo de frente (T_1) a partir de una recta virtual que cruza los puntos correspondientes al 30 % y 90 % del valor cresta en la zona de ascenso de la señal.

$$T_1 = \frac{t_{90} - t_{30}}{0,6} \quad (1.2)$$

Donde:

T_1 : tiempo de frente.

t_{90} : instante de tiempo en el que la tensión alcanza el 90 % de U_t .

t_{30} : instante de tiempo en el que la tensión alcanza el 30 % de U_t .

Para establecer el origen temporal de la onda, se calcula el origen virtual (O_1), proyectando la recta definida anteriormente hasta su intersección con el eje de abscisas ($U = 0$).

$$O_1 = t_{30} - 0,5 \cdot (t_{90} - t_{30}) \quad (1.3)$$

Donde: O_1 : origen virtual de la onda de impulso.

El tiempo de cola (T_2) se define como el intervalo temporal entre el origen virtual y el instante en que la tensión en la cola de la onda decrece al 50 % del valor cresta.

$$T_2 = t_{50} - O_1 \quad (1.4)$$

Donde: T_2 : tiempo de cola. t_{50} : instante de tiempo en la caída de la señal donde la amplitud equivale al 50 % de U_t .

La sobreoscilación de alta frecuencia en el pico de la señal (*overshoot*), si se encuentra presente, se cuantifica relacionando el pico de la curva de tensión de ensayo con el máximo de la curva base.

$$OS = \left(\frac{U_e - U_b}{U_e} \right) \cdot 100 \% \quad (1.5)$$

Donde:

OS : magnitud de la sobreoscilación en porcentaje.

U_e : valor pico absoluto de la curva de tensión de ensayo.

U_b : valor máximo absoluto de la curva base $u_m(t)$.

1.2 Filtrado de señales adquiridas

El tratamiento de los datos crudos inicia con la remoción del *offset* de corriente continua propio del sistema de medición. El algoritmo extrae un bloque de muestras previas al evento de disparo (*pre-trigger*) para calcular la media aritmética del ruido de fondo, restando esta constante a la totalidad de las muestras de la curva registrada.

Para la remoción del ruido electromagnético de alta frecuencia inherente al ensayo, se implementa un filtro digital de respuesta impulsional infinita (IIR) con coeficientes calculados paramétricamente según las indicaciones que prescribe la norma IEC 60060-1 [X]. La función de transferencia del filtro se define por la siguiente ecuación de diferencia:

$$y[i] = b_0 \cdot x[i] + b_1 \cdot x[i - 1] + a_1 \cdot y[i - 1] \quad (1.6)$$

$$\begin{aligned}b_0 &= b_1 = \frac{c}{1+c} \\a_1 &= \frac{1-c}{1+c} \\c &= \tan\left(\frac{\pi \cdot T_S}{\sqrt{d}}\right)\end{aligned}$$

Donde:

$$d = 2,2 \times 10^{-12}$$

T_S : Intervalo de muestreo usado al registrar la señal.

$x[i]$: Arreglo de muestras de entrada al filtro.

$y[i]$: Arreglo de muestras de salida del filtro.

A fines de evitar la distorsión del retardo de grupo (fase no lineal) que introducen los filtros IIR estándar, se emplea la técnica de filtrado de fase cero mediante la función `scipy.signal.filtfilt` de la biblioteca SciPy. Este método procesa el vector de la curva residual bidireccionalmente (hacia adelante y hacia atrás), anulando el corrimiento de fase temporal y garantizando la fidelidad morfológica de la onda.

1.3 Interpolación de datos

Como se vio anteriormente, el osciloscopio digital discretiza verticalmente la señal. Esta cuantización, sumada a la frecuencia de muestreo finita, imposibilita que los umbrales de tensión (30 %, 50 % y 90 %) coincidan exactamente con las muestras discretas de la matriz extraída. Para sortear esta limitación, el motor DSP implementa un algoritmo de interpolación lineal que aproxima el cruce de umbrales entre dos muestras subyacentes.

$$t_x = t_1 + (V_x - V_1) \cdot \frac{t_2 - t_1}{V_2 - V_1} \quad (1.7)$$

Donde:

t_x : instante de tiempo interpolado correspondiente al umbral de tensión buscado (V_x).
(t_1, V_1) y (t_2, V_2): coordenadas de tiempo y tensión de las muestras discretas inmediatamente anterior y posterior al umbral V_x .

1.4 Validación algorítmica mediante patrón TDG (IEC 61083-2)

La validación técnica del motor matemático se realiza inyectando vectores de datos de prueba (TDG - *Test Data Generator*) certificados bajo la norma de validación de software IEC 61083-2.

El programa procesa estos archivos de datos crudos estandarizados, ejecutando de

forma ciega las rutinas de *fitting*, filtrado IIR bidireccional, interpolación y detección de umbrales. Los parámetros calculados (U_t , T_1 , T_2 y OS) se contrastan algorítmicamente contra los valores nominales y los límites de aceptación exigidos por la norma.

Para certificar el correcto funcionamiento de la implementación DSP en Python, la matriz de comparación debe arrojar un error relativo porcentual inferior a los márgenes tolerados para equipos de medición e instrumentación clase 1 y clase 2.

Tabla 1.1: Comparativa de variables de validación para TDG IEC 61083-2

Parámetro	Símbolo	Tolerancia máxima admitida
Tensión de ensayo	U_t	$\pm 1 \%$
Tiempo de frente	T_1	$\pm 5 \%$
Tiempo de cola	T_2	$\pm 5 \%$
Sobreoscilación	OS	$\pm 2 \%$

1.5 Descripción funcional del software de procesamiento

1.5.1 Clase `LightningImpulseAnalyzer`

La clase `LightningImpulseAnalyzer` constituye el núcleo del motor de procesamiento digital de señales (DSP) del software. Su función principal es encapsular el vector unidimensional de *raw data* adquirido por el osciloscopio y ejecutar secuencialmente las rutinas matemáticas para la extracción paramétrica bajo la norma IEC 60060-1 y la normativa de validación IEC 61083-2. Maneja internamente atributos de estado para almacenar las matrices procesadas en cada etapa (señal con eliminación de *offset*, curva normalizada, curva base teórica, residuo de alta frecuencia y señal filtrada), así como los hiperparámetros de regresión no lineal y el diccionario contenedor de los resultados finales.

- Define la estructura de datos que soporta las operaciones vectorizadas mediante la librería NumPy.
- Gestiona la memoria almacenando vectores intermedios para evitar el recálculo durante la ejecución secuencial.
- Orquesta la ejecución de tuberías de procesamiento (*pipelines*) diferenciadas según el tipo de impulso (completo o cortado en la cola).

Método `__init__`

Inicializa la instancia de análisis, configurando el entorno matemático y validando las magnitudes de entrada fundamentales para la discretización temporal.

- Ejecuta validaciones de control sobre el periodo de muestreo y el factor de peso de la regresión (`sigma_fit`), lanzando excepciones si los valores son inválidos.
- Convierte la lista de datos crudos en un vector de tipo `numpy.ndarray`.
- Construye el eje temporal discretizado (`time_axis`) multiplicando los índices del vector por el periodo de muestreo.
- Declara e inicializa en un estado nulo (`None`) los atributos de clase y el diccionario de resultados.

Método `_remove_offset`

Remueve el componente de corriente continua (ruido de fondo u *offset*) de la señal registrada mediante el análisis probabilístico de la ventana de *pre-trigger*.

- Identifica el índice del valor máximo absoluto de la señal.
- Extrae el 30 % inicial de las muestras del frente de onda para calcular su media aritmética y desviación estándar.
- Identifica la muestra exacta donde la señal supera el ruido de fondo (umbral de 5 desviaciones estándar).
- Aísla el vector de ruido de fondo puro, calcula su valor medio y lo resta a la totalidad de la curva cruda para generar el vector `zeroed_curve`.

Método `_polarity_normalization`

Establece la polaridad de la onda de impulso y genera un vector de trabajo en valores absolutos para simplificar el procesamiento matemático posterior.

- Evalúa el signo de la amplitud máxima (U_e) sobre la curva sin *offset*.
- Define un factor multiplicador (1,0 o -1,0) según la polaridad detectada.
- Multiplica la curva por dicho factor para obtener el vector absoluto `zeroed_curve_abs` y extrae el valor pico real.

Método `_normalize_waveform`

Escala la magnitud vertical del vector absoluto al rango paramétrico de 0 a 1.

- Divide todos los elementos del vector `zeroed_curve_abs` por su valor pico máximo.

- Almacena el resultado en el atributo `norm_voltage`, utilizado posteriormente para comparaciones relativas entre impulsos.

Método `_find_limit_index`

Método estático de búsqueda de índices diseñado para identificar la intersección de la señal con un umbral de tensión específico, iterando en direcciones opuestas según la región de la onda.

- En modo *front*: invierte el arreglo y busca la primera muestra que caiga por debajo del umbral, corrigiendo el índice al sistema de referencia original.
- En modo *tail*: busca progresivamente desde el pico hacia adelante la primera muestra inferior al umbral.
- Implementa manejo de excepciones en caso de que la señal no alcance los umbrales exigidos.

Método `_cutting_signal`

Segmenta la curva de impulso, aislando exclusivamente la región de interés morfológico requerida para la regresión matemática.

- Llama al método de búsqueda de índices para localizar el nivel del 20 % de tensión en el frente y el 40 % en la cola.
- Genera vectores recortados (*slices*) de tensión y tiempo comprendidos estrictamente entre estos dos límites.

Método `_double_exponential_func`

Método estático que define el modelo matemático no lineal de la curva base de tensión, de acuerdo a la ecuación paramétrica de doble exponencial.

- Recibe el vector de tiempo discreto y los parámetros independientes (U, τ_1, τ_2, t_d).
- Implementa una máscara lógica ($dt \geq 0$) para evitar divergencias matemáticas y números complejos en la función exponencial.
- Evalúa la ecuación y retorna el vector numérico resultante.

Método `_fit_base_curve`

Ejecuta el algoritmo de regresión no lineal de optimización iterativa para encontrar los parámetros óptimos de la doble exponencial que mejor representen la señal adquirida.

- Define las semillas iniciales (*initial guesses*) para los parámetros matemáticos.
- Construye un vector de pesos (*sigma*), asignando el factor `sigma_fit` en las proximidades del pico para forzar al algoritmo a reducir el error de *fitting* en dicha zona crítica.
- Ejecuta la función `scipy.optimize.curve_fit` (basada en el algoritmo de Levenberg-Marquardt).
- Almacena los parámetros óptimos convergentes en el diccionario `fitted_params`.

Método `_construct_base_curve`

Reconstruye la curva base analítica completa ($u_m(t)$) evaluada sobre todo el eje temporal del ensayo.

- Invoca al método `_double_exponential_func` pasando el eje temporal original completo y los parámetros optimizados.
- Extrae y almacena el valor máximo absoluto del modelo matemático ajustado (U_b).

Método `_calculate_residual_curve`

Aísla la sobreoscilación y el ruido de alta frecuencia electromagnética inherentes a la medición.

- Ejecuta la sustracción vectorial restando la curva teórica (`base_curve`) a la curva empírica registrada en valor absoluto (`zeroed_curve_abs`).

Método `_create_digital_filter`

Sintetiza los coeficientes recursivos para un filtro digital de respuesta impulsional infinita (IIR) diseñado según la ecuación de diferencia de la norma IEC 60060-1.

- Inicializa la constante temporal del filtro en $d = 2,2 \times 10^{-12} \text{ s}^2$.

- Calcula la constante intermedia c basada en el periodo de muestreo.
- Determina los coeficientes analíticos de transferencia b_0 , b_1 y a_1 , retornando las matrices de numerador y denominador requeridas por SciPy.

Método `_filter_to_residual`

Aplica el filtro digital de amortiguamiento a la curva residual aislada en el paso anterior.

- Obtiene los coeficientes del filtro IIR.
- Implementa un filtrado bidireccional (*zero-phase filtering*) mediante la función `scipy.signal.filtfilt`, anulando el retraso de fase de la curva residual.

Método `_construct_test_voltage_curve`

Sintetiza la curva de tensión de ensayo definitiva sobre la cual se medirán los parámetros temporales y de amplitud.

- Suma la curva base teórica (`base_curve`) a la curva residual filtrada (`filtered_residual`).■
- Restaura el signo algebraico original de la onda aplicando el factor de polaridad.
- Extrae el valor cresta definitivo de la onda de ensayo (U_t) y normaliza la curva final.

Método `_linear_interpolation`

Método estático que implementa el algoritmo de interpolación lineal para aumentar la precisión en la detección del cruce de umbrales sobre el eje de tiempo continuo.

- Determina los pares de coordenadas bidimensionales de las muestras adyacentes al índice discreto más cercano al nivel de tensión objetivo.
- Evalúa la ecuación de la recta para calcular y retornar el instante temporal subyacente de alta resolución.

Método `_calc_front_parameters`

Determina los parámetros característicos de la zona de ascenso de la onda (Tiempo de frente y origen virtual).

- Localiza los índices discretos correspondientes a los umbrales del 30 % y 90 % de U_t .
- Interpola los tiempos exactos t_{30} y t_{90} .
- Calcula el tiempo de frente (T_1) aplicando la constante de 0,6 (inversa de 1,67) normativa.
- Proyecta el origen virtual (O_1) extrapolando linealmente la recta de frente hasta tensión nula.

Método `_calc_tail_parameter`

Determina el parámetro característico de la zona de descenso de la onda (Tiempo de cola).

- Localiza el índice discreto y el tiempo interpolado correspondiente a la caída del 50 % de U_t (t_{50}).
- Resta el origen virtual O_1 al instante t_{50} calculado para obtener el valor definitivo de T_2 .

Método `_calculate_parameters`

Orquesta la extracción de parámetros paramétricos globales e inyecta los resultados en el diccionario de la clase.

- Extrae el instante del pico máximo de la tensión de ensayo.
- Cuantifica porcentualmente la sobreoscilación de alta frecuencia (OS) comparando el valor pico discreto frente al máximo de la curva matemática de ajuste.
- Ejecuta los métodos de cálculo de frente y evalúa la variable `impulse_type` para bifurcar la lógica de cálculo del tiempo de cola, dependiendo de si el transitorio decae de forma natural o fue cortado por un espinterómetro.

Método `_find_time_lag`

[Específico para impulsos cortados] Analiza la desviación temporal entre la captura de una onda cortada y una onda completa de referencia capturada previamente.

- Interpola los instantes exactos en los que ambas ondas normalizadas cruzan los umbrales del 30 %, 50 % y 80 %.

- Calcula la diferencia de tiempo punto a punto y determina el desfase medio (t_L).

Método `_adjust_time_lag`

[Específico para impulsos cortados] Desplaza rígidamente el eje temporal de la onda capturada para sincronizar su frente geométrico con el impulso de referencia.

- Suma escalarmente el desfase calculado (t_L) al vector de eje de tiempo original.

Método `_find_deviation_point`

[Específico para impulsos cortados] Detecta algorítmicamente el punto morfológico en el cual la onda sometida a evaluación colapsa y se desvía de su trayectoria natural.

- Interpola el frente de la señal analizada mapeándolo sobre el eje temporal sincronizado de la señal de referencia.
- Ejecuta una sustracción punto a punto en la región posterior al pico y evalúa contra un umbral de tolerancia del 2 %.
- Actualiza la bandera de estado `impulse_type` y almacena el índice exacto de la bifurcación.

Método `_select_data_up_to_deviation`

[Específico para impulsos cortados] Purga la información de alta frecuencia posterior al corte del aislamiento.

- Realiza un *slicing* de los vectores de tensión y tiempo utilizando el índice de desviación.
- Obliga a realizar copias en memoria profunda (`.copy()`) para aislar la porción válida del frente de onda de cualquier alteración en los vectores originales.

Método `_find_amplitude_ratio`

[Específico para impulsos cortados] Determina la relación de escala en amplitud entre el ensayo a tensión plena (cortado) y el ensayo a tensión reducida (referencia).

- Promedia la tensión real contenida en el intervalo de mayor linealidad (entre el 30 % y el 80 % del frente).

- Calcula la división entre la media de la señal bajo ensayo y la media de la señal de referencia, obteniendo el escalar E .

Método `_scale_base_curve`

[Específico para impulsos cortados] Evita el algoritmo de Levenberg-Marquardt al no disponer de una cola de onda válida, generando la curva teórica a partir del modelo de la onda de referencia.

- Extrae los parámetros matemáticos temporales (τ_1, τ_2, t_d) del objeto de referencia.
- Multiplica la amplitud paramétrica de referencia (U_{ref}) por el factor de escala (E).
- Reconstruye el vector base del impulso cortado evaluando la ecuación exponencial con los parámetros clonados.

Método `_find_chopping_instant`

[Específico para impulsos cortados] Estima el instante analítico del corte del impulso de forma análoga al cálculo del tiempo de frente de un impulso estándar.

- Calcula la derivada discreta (gradiente) del colapso post-pico para encontrar el punto de máxima pendiente.
- Identifica el voltaje de colapso de aislamiento ($U_{collapse}$).
- Extrae los tiempos correspondientes al nivel del 70 % y el 10 % referidos al voltaje de colapso, aplicándoles regresión lineal para extrapolar el momento virtual de corte ($T_{cutting_moment}$).

Método `ref_lightning_impulse`

Orquesta el *pipeline* de procesamiento integral diseñado para aislar los parámetros de una onda de impulso completo (ensayo a tensión reducida de calibración).

- Ejecuta de manera secuencial los trece métodos privados necesarios para llevar el transitorio desde formato *raw data* hasta la extracción de los indicadores paramétricos.

Método `lightning_impulse`

Orquesta el *pipeline* de procesamiento adaptativo diseñado para transitorios capturados durante el ensayo a tensión plena de la máquina o aislador.

- Requiere y valida la inyección de la instancia del analizador de referencia calculada previamente.
- Ejecuta los métodos de sincronización y alineamiento temporal.
- Determina en tiempo de ejecución si el transitorio es de tipo completo o cortado, bifurcando el flujo lógico de construcción de la curva base.
- Finaliza ejecutando la sustracción del residuo, filtrado pasabajos y cálculo paramétrico final en función del tipo de onda.